

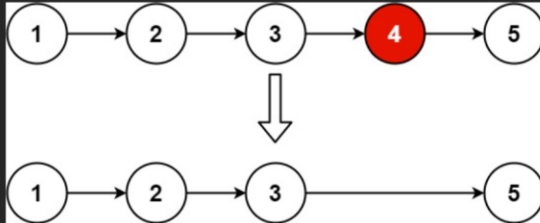
LCR 021. 删除链表的倒数第 N 个结点

已解答

中等 相关标签 相关企业 Aa

给定一个链表，删除链表的倒数第 n 个结点，并且返回链表的头结点。

示例 1:



输入: head = [1,2,3,4,5], n = 2
输出: [1,2,3,5]

示例 2:

输入: head = [1], n = 1
输出: []

C++ 智能模式

```
1 /**
2  * Definition for singly-linked list.
3  * struct ListNode {
4  *     int val;
5  *     ListNode *next;
6  *     ListNode() : val(0), next(nullptr) {}
7  *     ListNode(int x) : val(x), next(nullptr) {}
8  *     ListNode(int x, ListNode *next) : val(x), next(next) {}
9  * };
10 */
11 class Solution {
12 public:
13     ListNode* removeNthFromEnd(ListNode* head, int n) {
14         // 定义一个哑结点
15         // ListNode(int x, ListNode *next) : val(x), next(next) {}
16         ListNode* dummy = new ListNode(0, head);
17         // 定义一个栈
18         stack<ListNode*> stk;
19         ListNode* cur = dummy;
20         // 循环入栈
21         while(cur){
22             stk.push(cur);
23             cur = cur->next;
24         }
25         for(int i = 0; i < n; i++) stk.pop();
26         ListNode* prev = stk.top();
27         prev->next = prev->next->next;
28         ListNode* ans = dummy->next;
29         delete dummy;
30         return ans;
31     }
32 };
```

238. 除了自身以外数组的乘积

已解答

中等 相关标签 相关企业 提示 Aa

给你一个整数数组 `nums`，返回 数组 `answer`，其中 `answer[i]` 等于 `nums` 中除了 `nums[i]` 之外其余各元素的乘积。

题目数据 保证 数组 `nums` 之中任意元素的全部前缀元素和后缀的乘积都在 32 位 整数范围内。

请 不要使用除法，且在 $O(n)$ 时间复杂度内完成此题。

示例 1:

输入: nums = [1,2,3,4]
输出: [24,12,8,6]

示例 2:

输入: nums = [-1,1,0,-3,3]
输出: [0,0,9,0,0]

C++ 智能模式

```
1 class Solution {
2 public:
3     vector<int> productExceptSelf(vector<int>& nums) {
4         int len = nums.size();
5         if(len == 0) return {};
6         vector<int> ans(len);
7         ans[0] = 1;
8         for(int i = 1; i < len; i++){
9             ans[i] = ans[i-1] * nums[i-1];
10        }
11        // 最开始右边没有元素
12        int right = 1;
13        for(int i = len-1; i >= 0; i--){
14            ans[i] = ans[i] * right;
15            right *= nums[i];
16        }
17        return ans;
18    }
19 };
```

209. 长度最小的子数组

已解答

中等 相关标签 相关企业 Aa

给定一个含有 n 个正整数的数组和一个正整数 `target`。

找出该数组中满足其总和大于等于 `target` 的长度最小的子数组 `[numsi, numsi+1, ..., numsj, numsj+1]`，并返回其长度。如果不存在符合条件的子数组，返回 0。

示例 1:

输入: target = 7, nums = [2,3,1,2,4,3]
输出: 2
解释: 子数组 [4,3] 是该条件下的长度最小的子数组。

示例 2:

输入: target = 4, nums = [1,4,4]
输出: 1

示例 3:

输入: target = 11, nums = [1,1,1,1,1,1,1,1]
输出: 0

C++ 智能模式

```
1 class Solution {
2 public:
3     int minSubArrayLen(int target, vector<int>& nums) {
4         // 数组长度
5         // j-1 pre[j] - pre[j-1] >= k; pre[i] - k >= pre[j-1]
6         int n = nums.size();
7         vector<int> pre(n+1, 0);
8         // 算出前缀和
9         for (int i = 0; i < n; i++) {
10             pre[i+1] = pre[i] + nums[i];
11         }
12
13         int ans = INT_MAX;
14         for (int i = 1; i <= n; i++) {
15             int need = pre[i] - target;
16             // 二分查找: 第一个大于 need 的位置, 往前一位就是最大的满足条件的 j
17             auto it = upper_bound(pre.begin(), pre.begin() + i, need);
18             int j = it - pre.begin() - 1;
19             if (j >= 0) {
20                 ans = min(ans, i - j);
21             }
22         }
23         return ans == INT_MAX ? 0 : ans;
24     }
25 };
```

303. 区域和检索 - 数组不可变

已解答

简单 相关标签 相关企业 Aa

给定一个整数数组 `nums`，处理以下类型的多个查询：

1. 计算索引 `left` 和 `right`（包含 `left` 和 `right`）之间的 `nums` 元素的和，其中 `left <= right`

实现 `NumArray` 类：

- `NumArray(int[] nums)` 使用数组 `nums` 初始化对象
- `int sumRange(int left, int right)` 返回数组 `nums` 中索引 `left` 和 `right` 之间的元素的总和，包含 `left` 和 `right` 两点（也就是 `nums[left] + nums[left + 1] + ... + nums[right]`）

示例 1：

输入：
["NumArray", "sumRange", "sumRange", "sumRange"]

```
1 class NumArray {
2 public:
3     vector<int> sums;
4     NumArray(vector<int>& nums) {
5         int len = nums.size();
6         sums.resize(len+1);
7         for(int i = 0; i<len; i++){
8             sums[i+1] = sums[i] + nums[i];
9         }
10    }
11
12    int sumRange(int left, int right) {
13        //从[left,right]的数
14        return sums[right+1] - sums[left];
15    }
16 };
17
18 /**
19  * Your NumArray object will be instantiated and called as such:
20  * NumArray* obj = new NumArray(nums);
21  * int param_1 = obj->sumRange(left,right);
22  */
```

1413. 逐步求和得到正数的最小值

已解答

简单 相关标签 相关企业 提示 Aa

给你一个整数数组 `nums`。你可以选定任意的正数 `startValue` 作为初始值。

你需要从左到右遍历 `nums` 数组，并将 `startValue` 依次累加上 `nums` 数组中的值。

请在确保累加和始终大于等于 1 的前提下，选出一个最小的正数作为 `startValue`。

示例 1：

输入：nums = [-3,2,-3,4,2]
输出：5

解释：如果你选择 `startValue = 4`，在第三次累加时，和小于 1。

累加求和

startValue = 4	startValue = 5	nums
(4 -3) = 1	(5 -3) = 2	-3
(1 +2) = 3	(2 +2) = 4	2
(3 -3) = 0	(4 -3) = 1	-3
(0 +4) = 4	(1 +4) = 5	4
(4 +2) = 6	(5 +2) = 7	2

```
1 class Solution {
2 public:
3     int minStartValue(vector<int>& nums) {
4         //前缀和pre[i] 前i个数
5         int len = nums.size();
6         if(len == 0)return 0;
7         vector<int>pre(len+1,0);
8         for(int i = 1; i<=len; i++){
9             pre[i] = pre[i-1] + nums[i-1];
10        }
11        int startValue = INT_MAX;
12        for(auto preNum:pre){
13            startValue = min(startValue,preNum);
14        }
15        return 1-startValue;
16    }
17 };
```

560. 和为 K 的子数组

已解答

中等 相关标签 相关企业 提示 Aa

给你一个整数数组 `nums` 和一个整数 `k`，请你统计并返回该数组中和为 `k` 的子数组的个数。

子数组是数组中元素的连续非空序列。

示例 1：

输入：nums = [1,1,1], k = 2
输出：2

示例 2：

输入：nums = [1,2,3], k = 3
输出：2

C++ 智能模式

```
1 class Solution {
2 public:
3     int subarraySum(vector<int>& nums, int k) {
4         //哈希表用于查元素
5         unordered_map<int,int>mp;
6         mp[0] = 1;
7         //前缀和
8         int pre = 0;
9         //结果
10        int ans = 0;
11
12        for(auto num:nums){
13            pre += num;
14            if(mp.find(pre-k)!=mp.end()){
15                //如果找到了
16                ans += mp[pre-k];
17            }
18            mp[pre]++;
19        }
20        return ans;
21    }
22 };
```

题目描述题解提交记录

3702. 按位异或非零的最长子序列

已解答

中等相关标签相关企业提示A+

给你一个整数数组 `nums`。

返回 `nums` 中按位异或 (XOR) 计算结果非零的最长子序列的长度。如果不存在这样的子序列，返回 0。

子序列是一个非空数组，可以通过从原数组中删除一些或不删除任何元素（不改变剩余元素的顺序）派生而来。

示例 1:

输入: `nums = [1,2,3]`

输出: 2

解释:

最长子序列之一是 `[2, 3]`。按位异或计算为 `2 XOR 3 = 1`，它是非零的。

示例 2:

输入: `nums = [2,3,4]`

C++智能模式

```
1 class Solution {
2 public:
3     bool isAllZero(vector<int>& nums){
4         for(auto num:nums){
5             if(num != 0)return false;
6         }
7         return true;
8     }
9
10    bool flag = false;
11    int longestSubsequence(vector<int>& nums) {
12        //子序列问题 全部元素异或 如果不等于0 nums自身就是个子序列 如果 = 0
13        //1. 元素全为0
14        //2. 至少一个元素不是0
15        //那就说明 其他所有元素异或之后 再与 这至少一个这个元素异或 == 0
16        //那么其他哪些元素异或 答案并不等于0 所以 else*x = 0 所以长度为n-1
17        int len = nums.size();
18        if(len == 0)return 0;
19        int res = 0;
20        for(auto num:nums){
21            res ^= num;
22        }
23        if(res != 0){
24            return len;
25        }else{
26            flag = isAllZero(nums);
27        }
28        return flag?0:len-1;
29    };
}
```